Practical 2 - Matrix computation

Xiru Zhang

Year 2023/2024

Contents

1 Exercise 6 Image compression using the SVD

1.1 Algorithm for image compression

Algorithm 1 SVD-Based Image Compression

1: Load the image matrix from a file: `clown.mat`
2: Perform Singular Value Decomposition on the image matrix X: $[U, S, V] \leftarrow \text{svd}(X)$
3: Select the number of singular values to retain k
4: Construct the approximate image matrix X_k: $U_k \leftarrow U(:, 1:k); S_k \leftarrow S(1:k, 1:k); V_k \leftarrow V(:, 1:k); X_k \leftarrow U_k \times S_k \times V_k'$
5: Display the original and compressed images
6: Calculate the compression ratio
7: Calculate the reconstruction error
8: Output the results

We choose the number of singular values to retain $k = 100$, aiming to balance between the level of compression and the quality of the reconstructed image. With this selection, the compressed image, as shown in Figure 1, exhibits certain levels of degradation, but still retains the essential features of the original image. The compression ratio achieved is 1.2284, reducing the data size needed to represent the image, while the reconstruction error is measured to be 511.6505, which indicates the degree of deviation from the original image.

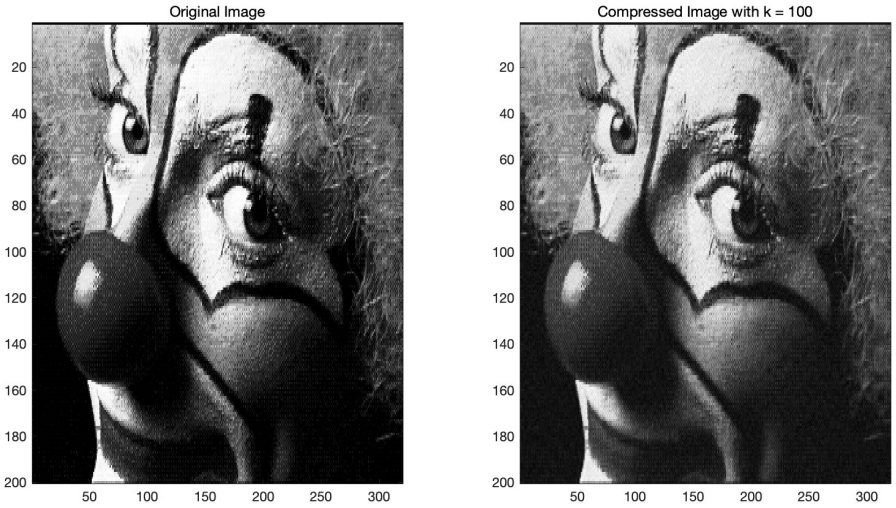

Figure 1: Comparison of original image and SVD (k=100)

1.2 Compression radio

Define a compression ratio to measure the quality of the compression. The compression ratio (CR) can be defined as the ratio of the size of the original image ($\text{Size}_{\text{original}}$) to the size of the compressed image ($\text{Size}_{\text{compressed}}$):

$$CR = \frac{\text{Size}_{\text{original}}}{\text{Size}_{\text{compressed}}} \tag{1}$$

For an image represented by a matrix X of size $m \times n$, and the compressed image obtained by retaining the first k singular values and corresponding vectors, the sizes can be expressed as follows:

- The size of the original image is $m \times n$.
- The size of the compressed image is the sum of the sizes of the matrices storing the significant components: $m \times k$ for the U_k matrix, k for the diagonal Σ_k matrix, and $n \times k$ for the V_k^T matrix. Thus, $\text{Size}_{\text{compressed}} = m \times k + k + n \times k$.

Therefore, the compression ratio is given by:

$$CR = \frac{m \times n}{m \times k + k + n \times k} \tag{2}$$

A higher CR indicates a higher level of compression, but it may also lead to a loss in the quality of the reconstructed image, which can be measured by the reconstruction error. The trade-off between compression ratio and reconstruction quality needs to be balanced for optimal results.

2 Exercise 7 Deblurring images using the SVD

2.1 Deriving the SVD Solution for Linear Deblurring

Given that the blurring matrix K can be factorized using its Singular Value Decomposition (SVD) as $K = U\Sigma V^T$, where U and V are orthogonal matrices and Σ is a diagonal matrix containing the singular values of K. The problem defined by $g = Kf + n$ aims to find f that minimizes the least squares error $\|g - Kf\|_2^2$. Ignoring the noise term n for now, we focus on solving $g = Kf$. Multiplying both sides of $g = Kf$ by V and using the orthogonality property $V^TV = I$, where I is the identity matrix, we get $V^Tg = V^TKf$. Substituting K with its SVD, we have $V^Tg = V^TU\Sigma V^Tf$. Since V is orthogonal, $V^TV = I$ and V^TU is also orthogonal, hence $V^TU = I$. This simplifies the equation to $V^Tg = \Sigma V^Tf$. Now, multiplying both sides by Σ^{-1}, assuming Σ is invertible, we get $\Sigma^{-1}V^Tg = V^Tf$. Finally, multiplying both sides by V, we obtain $V\Sigma^{-1}V^Tg = f$. Putting it all together, the solution to $g = Kf$ is given by:

$$f^* = V\Sigma^{-1}U^Tg$$

This solution assumes that the singular values are non-zero and that the noise term n is negligible.

2.2 Deblurring image with SVD

The selection of the optimal number of singular values, denoted as p, for image deblurring was guided by a clarity evaluation function named `evaluate_image_clarity`. This function quantitatively assesses the clarity of an image based on the energy of its gradients. The rationale behind using gradient energy as a clarity metric stems from the observation that clearer images tend to exhibit higher edge and detail contrast, which translates to higher gradient magnitudes across the image.

This clarity evaluation function was pivotal in automating the selection of the p value that yields the best deblurring effect. By iterating over a range of p values and computing the clarity score for the deblurred image at each step, the p value corresponding to the lowest score (highest gradient energy) was identified as the optimal choice for image restoration. The clearest image is given by $p = 30$, as shown in 2

Listing 1: Debur with SVD function

```
function [deblurred_image, best_p] =
   deblur_image_with_svd(A, B, G)
    g_vector = G(:);
    [U_A, S_A, V_A] = svd(A);
    [U_B, S_B, V_B] = svd(B);

    % Set the range of p values
    p_values = 1:min([length(S_A), length(S_B), 30]);
    best_score = inf; % Assuming lower scores are better
    best_p = p_values(1);

    for p = p_values
        f_vector = zeros(size(g_vector));
        for i = 1:p
            for j = 1:p
                v_kron = kron(V_A(:,i), V_B(:,j));
                proj = dot(g_vector, v_kron);
                scaling_factor = 1 / (S_A(i,i) * S_B(j,j
                   ));
                u_kron = kron(U_A(:,i), U_B(:,j));
                f_vector = f_vector + scaling_factor *
                   proj * u_kron;
            end
        end
        temp_image = reshape(f_vector, size(G));
        score = evaluate_image_clarity(temp_image); %
           You need to define this function
        if score < best_score
            best_score = score;
            best_p = p;
```

```
            deblurred_image = temp_image;
        end
    end
end

function score = evaluate_image_clarity(image)
    % Define a simple clarity evaluation function, e.g.,
        the energy of image gradients
    gradX = abs(diff(image, 1, 2));
    gradY = abs(diff(image, 1, 1));
    score = -sum(gradX(:)) - sum(gradY(:)); % Negative
       sign because we want to maximize gradient energy
end
```

Figure 2: Deblurred image with best p = 30

3 Exercise 8 Google PageRank algorithm

3.1 Proof that A^T is a stochastic matrix

By definition, A_{ij} is either a fraction of non-negative quantities or $\frac{1}{n}$. Since p, g_{ij}, and c_j are non-negative and δ is a fraction of non-negative numbers, all elements of A are non-negative. Transposition does not change the element values, hence all elements of A^T are also non-negative.

For columns of A where $c_j \neq 0$:

$$\sum_{i=1}^{n} A_{ij} = \sum_{i=1}^{n} \left(\frac{pg_{ij}}{c_j} + \delta \right) = p \sum_{i=1}^{n} \frac{g_{ij}}{c_j} + n\delta = p\frac{c_j}{c_j} + n\frac{1-p}{n} = p + 1 - p = 1.$$

For columns of A where $c_j = 0$:

$$\sum_{i=1}^{n} A_{ij} = \sum_{i=1}^{n} \frac{1}{n} = \frac{n}{n} = 1.$$

Since the sum of the elements in each column of A is 1, the sum of the elements in each row of A^T is also 1. Thus, A^T is a stochastic matrix, satisfying the properties of non-negativity and row sums equal to 1. Hence 1 is a eigenvalue of A, thus $A1 = 1$. According to the Perron-Frobenius theorem, the largest eigenvalue of a non-negative matrix is at least as large as the absolute value of any other eigenvalue of the matrix. Since matrix A is non-negative and we know it has an eigenvalue of 1 (the eigenvalue corresponding to the vector of all ones), therefore 1 is the largest eigenvalue of A.

3.2 Proof

Given matrix A is nonnegative and A^T is a stochastic matrix, we proceed to show that there exists a nonnegative eigenvector x corresponding to the eigenvalue 1 using the Perron-Frobenius theorem.

Since A is nonnegative and A^T is stochastic, every column of A sums to 1. By the Perron-Frobenius theorem, there exists a largest eigenvalue λ of A which is positive, and the corresponding eigenvector x can be chosen to be non-negative. Furthermore, because A is stochastic, we have that $A1 = 1$, where 1 is a vector of all ones. Hence, 1 is an eigenvalue of A, and by the Perron-Frobenius theorem, it is the largest eigenvalue.

Now, let x be the nonnegative eigenvector corresponding to the eigenvalue 1. Since the sum of each column of A is 1, multiplying A by a vector of ones yields another vector of ones, which implies that the eigenvector x can be normalized such that its components sum up to 1. Therefore, we can write:

$$Ax = x$$
$$\sum_{i=1}^{n} x_i = 1$$

3.3 Compute x using the power method

The power method function returns NaN values for both the PageRank vector $\mathbf{x}$ and the largest eigenvalue λ. This is likely due to numerical instability issues, which are common when dealing with sparse matrices. In our case, the matrix G is sparse and was obtained from the 'surfer' function, which generates a web

link matrix from the given root page 'http://www.sorbonne-universite.fr/' with a depth of 15.

$$\mathbf{x} = \begin{bmatrix} \text{NaN} \\ \text{NaN} \\ \vdots \\ \text{NaN} \end{bmatrix}, \quad \lambda = \text{NaN} \tag{3}$$

The sparsity of the matrix G suggests that many web pages do not link to each other, which can result in zero columns in the matrix. When such a matrix is used in the power method without proper handling of zero columns, the division by zero during normalization leads to NaN values.

Listing 2: Power Method Implementation in MATLAB

```
function [x, lambda] = power_method(A, tol, max_iter)
    n = size(A,1); % The number of pages
    x = rand(n, 1); % Initialize x with random values
    x = x / norm(x, 1); % Normalize x to have norm 1
    lambda_old = 0;
    iter = 0;

    for iter = 1:max_iter
        y = A * x; % Multiply A with the current x
        lambda = norm(y, inf); % Approximate the largest
            eigenvalue
        x = y / lambda; % Update x
        % Check for convergence
        if abs(lambda - lambda_old) < tol
            break;
        end
        lambda_old = lambda;
    end
    x = x / sum(x); % Normalize x so that the sum is 1
end
```